SanskritNet: End-to-End Cross-Lingual Poetic Style Transfer

Shubham Joshi / SanskritNet33
DAU
Gandhinagar, India
202511048@dau.ac.in

Abstract

SanskritNet is an end-to-end deep learning web application designed to bridge the linguistic gap between classical Sanskrit and historical poetic dialects. Utilizing a two-stage process to extract semantic meaning and transfer it into a new poetic style, the system successfully generates Awadhi text matching the style of the *Ramcharitmanas*. By combining parameter-efficient AI training on lightweight models with a fully functioning web deployment, this project demonstrates a highly accessible and computationally efficient approach to preserving low-resource cultural heritage languages.

1 Motivation

Cultural Importance: Historical dialects like Tulsidas-style Awadhi embody a massive part of our cultural and literary heritage.

The AI Problem: These historical languages remain critically underrepresented in modern Artificial Intelligence (NLP) due to a severe scarcity of parallel training corpora.

The Translation Challenge: Directly translating an ancient language like Sanskrit into a localized historical dialect is extremely difficult for machine translation systems.

Project Goal: This project is motivated by the social and cultural imperative to create accessible, computationally efficient tools that can perform cross-lingual poetic style transfer. The ultimate goal is to bring classical literature to the general public without requiring massive, expensive supercomputers.

2 Problem Definition

The Core Task: The project addresses the challenge of building a system capable of converting classical Sanskrit verses into poetic Awadhi while retaining original semantic fidelity.

Why Standard AI Fails: Standard translation models fail because there is almost no direct Sanskrit-to-Awadhi data. Consequently, they often hallucinate or produce meaningless garbage tokens.

The Technical Challenge: The primary challenge lies in developing a resource-efficient architecture capable of running smoothly on a standard computer processor (CPU limit). The system must accurately map the semantic structures of well-documented languages to the strict syntactic and poetic constraints of a historical dialect.

3 Key Contributions

To solve these problems, the project introduces four major contributions:

- **Novel Two-Stage Pipeline:** The team built a hybrid system to bypass the lack of translation data. Stage 1 utilizes the Google Translate API as a semantic bridge (Sanskrit → Modern Hindi). Stage 2 uses a custom deep learning model for style transfer (Modern Hindi → Poetic Awadhi).
- **Curated Dataset:** The team manually collected a specialized dataset of over 500 authentic *Ramcharitmanas* verse triplets. The data was cross-validated for quality, with high-confidence pairs (overlap $\geq$ 0.4) upsampled to improve model robustness.
- **Efficient Fine-Tuning (PEFT):** The team applied Low-Rank Adaptation (LoRA) to train a lightweight foundation model (`mT5-small`). This created a highly specialized AI that runs easily on standard computers without catastrophic forgetting.
- **Robust Web Deployment:** The team built a complete, interactive Gradio web interface. It features bilingual support, text filters to catch AI garbage tokens, and a built-in fallback dictionary that perfectly translates famous phrases if the AI becomes confused.

4 High-Level Results and Analysis

The SanskritNet model was tested against the basic, untrained `mT5-small` baseline to prove its effectiveness.

4.1 Quantitative Results

The team utilized the **chrF++** scoring metric, which is vastly superior for evaluating heavily inflected scripts like Devanagari. The newly trained model showed a massive, statistically significant improvement over the baseline. While the basic model generated blank padding tokens or hallucinated modern text, SanskritNet proved it successfully learned Awadhi vocabulary and syntactic rhythm.

4.2 Qualitative Analysis

Qualitatively, the two-stage approach successfully prevented the AI from experiencing "catastrophic forgetting." Furthermore, for highly complex inputs where the model's confidence dropped, the custom safety rules immediately stepped in. The system intercepted bad outputs and utilized the fallback dictionary to provide historically accurate translations, ensuring the application remained highly resilient and never crashed during end-to-end use.